Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

Assignment No 1

Name: Soham Kulkarni
PRN: 124B1F056
Subject: Design and Analysis of Algorithms
Batch: A3
GitHub Link: https://github.com/SOHAM0007-CODER/DAA_ASSIGNMNET

1. Topic

Apply Merge Sort to arrange large record datasets by a numeric key. The warehouse version sorts stock quantities ascending; this implementation demonstrates the same algorithm on IPL player-team-season runs in descending order, with all records displayed.

2. Course Objective

- Understand the computational complexity of algorithms.
- Select a suitable algorithm design strategy for a practical problem.

3. Course Outcome

- Apply Divide and Conquer to a real cricket dataset.
- Analyze stable sorting, comparisons, time and auxiliary storage.

Theory

Merge Sort splits a range into two halves, recursively sorts each half and merges the sorted results. Choosing the left record on equal runs preserves stability. The implementation reads 2,617 player-team-season records, aggregated from regular IPL innings in 2008-2024.

Kaggle source: patrickb1912/ipl-complete-dataset-20082020 (dataset title: IPL Complete Dataset 2008-2024). Only batsman runs are summed; extras and super-over innings are excluded. A row is a season aggregate, not a unique career total. The CSV and source notes accompany the C++ file.

Working of the Algorithm

- 1. Read name, team, runs and season from IPLBatters.csv.
- 2. Split recursively, sort both halves and merge with the larger runs first.
- 3. Print every sorted record with rank, season and runs.
- 4. Report the record count, comparisons, merges and sorting-only elapsed time.

Complexity Analysis

T(N) = 2T(N/2) + O(N). There are logarithmically many recursion levels and linear merging work per level. Timing excludes file reading and output.

Time Complexity

Best Case: O(N log N).

Average Case: O(N log N).

Worst Case: O(N log N).

Space Complexity

O(N) peak auxiliary storage for merge buffers, plus O(log N) recursion. The input vector stores O(N) records.

Algorithm - Steps

- Step 1: Read the complete CSV and validate records.
- Step 2: Recursively split and merge using runs as the descending key.
- Step 3: Display every row and a sorting summary.

Pseudocode

```
MERGE_SORT(A, left, right):
    if left >= right: return
    mid = left + (right-left)/2
    MERGE_SORT(A,left,mid); MERGE_SORT(A,mid+1,right)
    merge halves, taking larger runs first
    on equal runs, take the left record first
```

Flow of Execution

START -> Read IPL CSV -> Divide and recurse -> Stable merge -> Print all 2,617 rows -> Summary -> END.

6. Example

Runs [45,12,78,23] split into [45,12] and [78,23]. Merging descending produces [78,45,23,12].

Merge Sort has predictable N log N work. Bubble Sort and basic Insertion Sort have quadratic average/worst cases; Insertion Sort can be linear on already-sorted input. Their implementations are not included in this program.

Conclusion

Merge Sort produces a stable descending order across all supplied batting records. Replacing player fields with inventory fields and reversing the comparison adapts it to the warehouse statement.